Module-V
Lecture-II
Syntax and Semantics of CTL

1. **Introduction**

In the previous lecture we have discussed three types of logic namely, propositional, predicate and temporal. It was also elaborated that among these three logic classes, temporal logic is a specification language, which is used to capture the timing modalities like next, future, past, etc. Temporal logic framework can be divided into different classes namely, Linear-time Temporal Logic (LTL), Computational Tree Logic (CTL), CTL* etc. The expressive power of these logic classes are different. In this lecture we will discuss the Computational Tree Logic (CTL), which is branching time logic.

A branching-time temporal logic represents time as a tree, rooted at the present instance of time and branching out into the future. The time is visualized as branching in nature which has many future possibilities but at any instant of time it will take one particular future direction depending on system behavior. Figure 1 shows the "branching" progress of time in different possible "braches" in future.

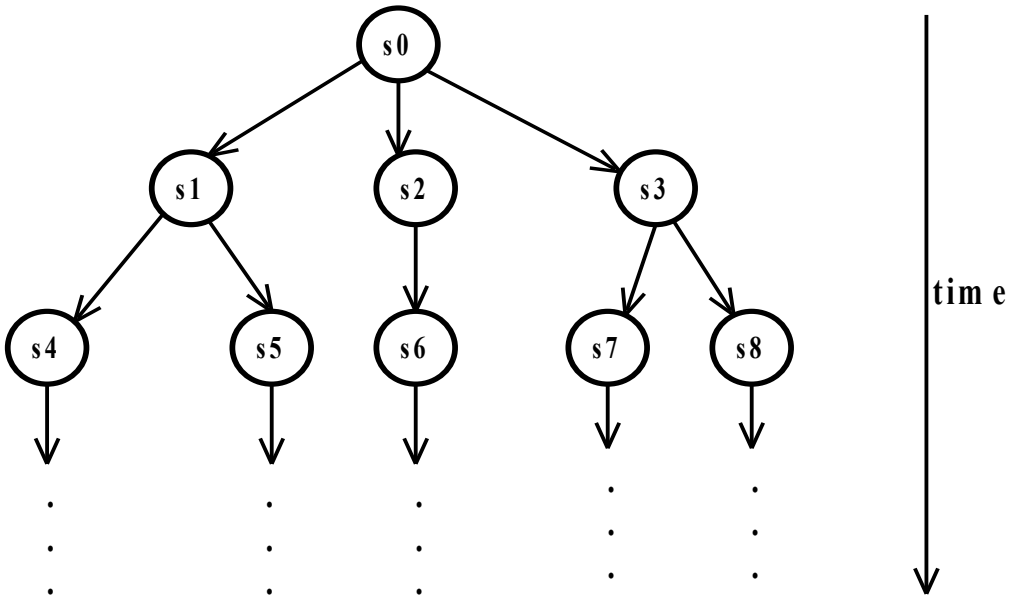


Figure 1. "Branching" progress of time

Branching-time logic allowing us to quantify statements over ALL paths and a SINGLE path from a state; the concept is shown in Figure 2.

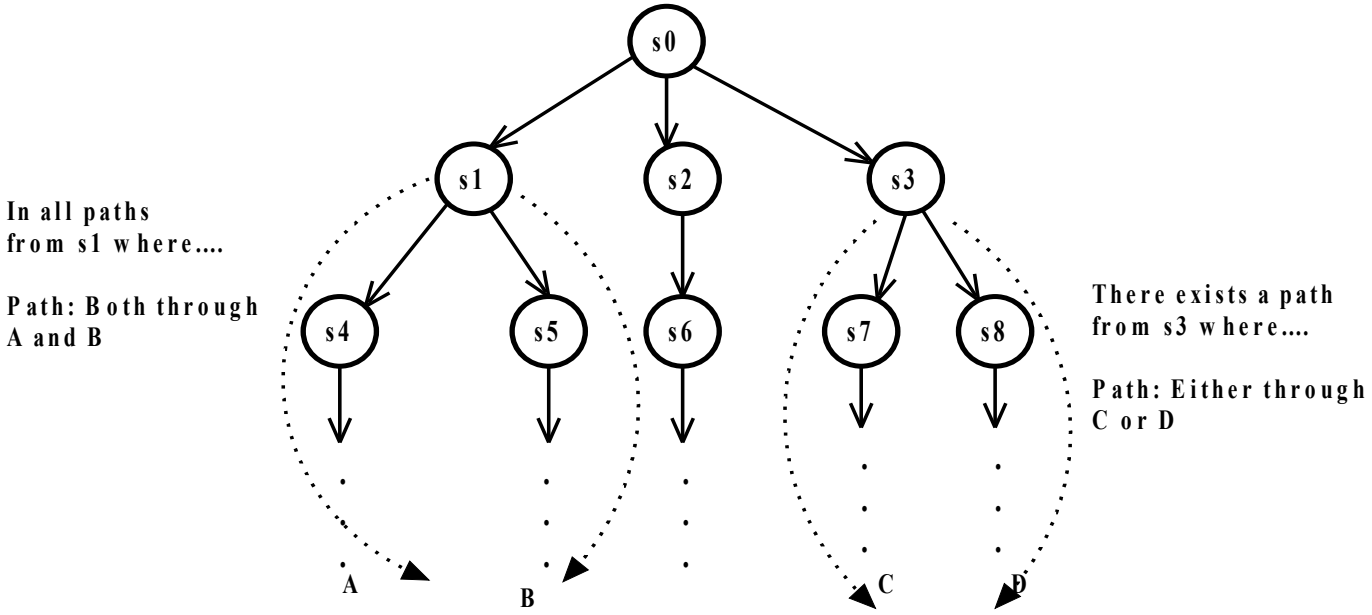


Figure 2. Statements over ALL paths and a SINGLE path in branching time temporal logic

2. **Syntax of CTL**

A CTL formula comprises

1. Atomic propositions such as *{p, q, r.....}*
2. Path Quantifiers *{A,E}*
    a. *A* : all paths starting from a given state.
    b. *E* : there exists at least one path from a given state.
3. Propositional logic operators such as AND ($\wedge$), OR ($\vee$), NOT ($\neg$)
4. Temporal operators *{X,F,G,U}*
    a. NE**X**T: next states of current state.
    b. **F**UTURE: any one of future states from the current state.
    c. **G**LOBAL: all future states from the current state.
    d. **U**NTIL: Some CTL formula holds until another CTL formula, from the current state.

X,F,G are unary operators and *U* is binary operator.

These temporal operators are illustrated using an example in Figure. 3.

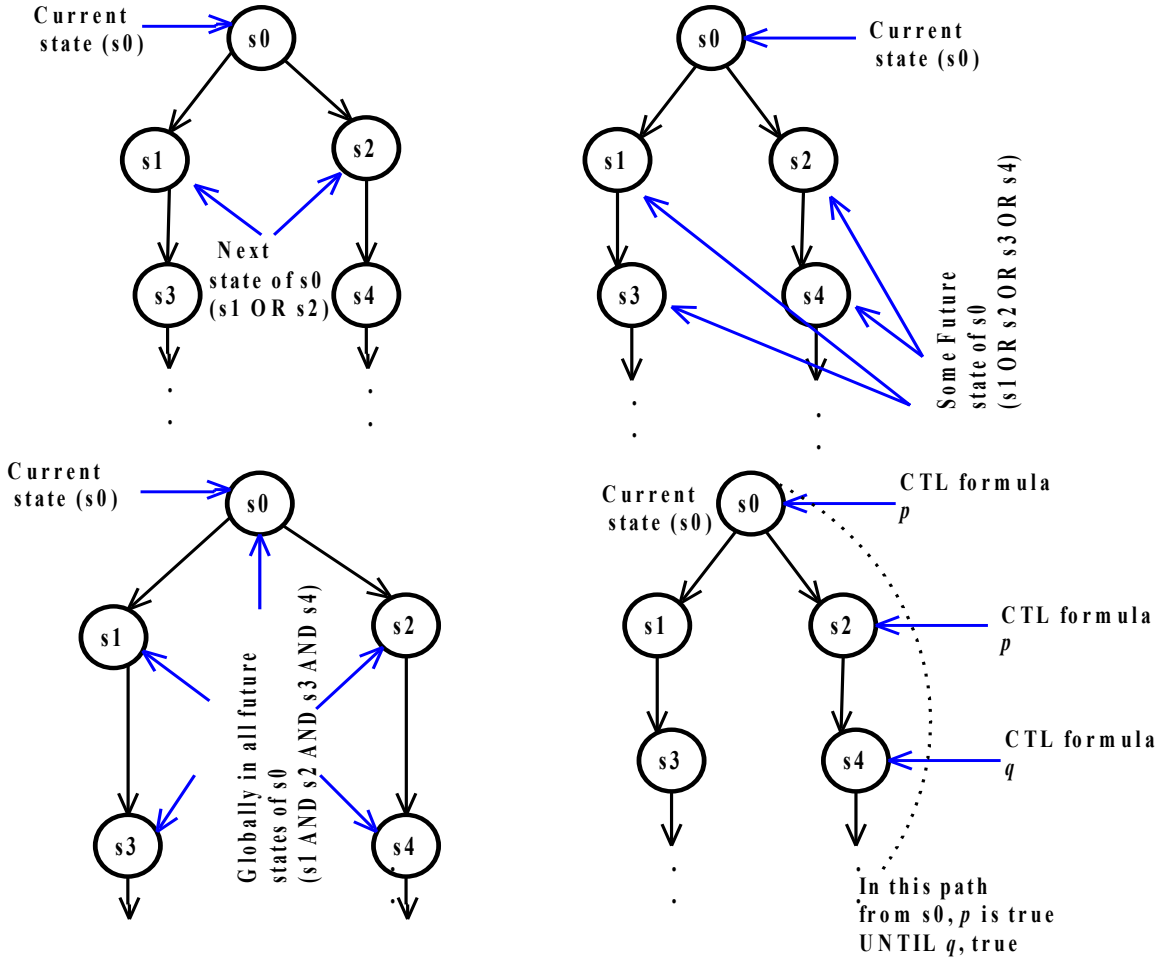


Figure 3. Illustration of basic CTL temporal operators using an example

We can define CTL formulas as below:

$\Phi ::= \bot \mid \top \mid P \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi) \mid AX\varphi \mid EX\varphi \mid AF\varphi \mid EF\varphi \mid AG\varphi \mid EG\varphi \mid A[\varphi\ U\ \varphi] \mid E[\varphi\ U\ \varphi];$

where

- The symbol $\top$ means truth value 'true' and symbol $\bot$ means truth value 'false'.
- *P* ranges over a set of atomic propositions

We can notice here that each of the CTL temporal connectives is a pair of symbols. The first element of the pair is either of *A* or *E*; A implies 'along All paths' (*inevitably*) and *E* means 'along at least (there Exists) one path' (*possibly*). The second element of the pair is *X, F, G,* or *U*, meaning 'neXt state,' 'some Future state,' 'all future states (Globally)' and Until, respectively. The syntax of CTL demands that each temporal operator must be preceded by a path quantifier. *X, F, G,* or *U* must be preceded by *A* or *E*. Similarly, *A* or *E* must be followed by *X, F, G,* or *U*. In $A[\varphi\ U\ \varphi]$, *A* is followed by *U* and in $E[\varphi\ U\ \varphi]$, *E* is followed by *U*.

Now we will explain the CTL formulas $AX\varphi \mid EX\varphi \mid AF\varphi \mid EF\varphi \mid AG\varphi \mid EG\varphi \mid A[\varphi\ U\ \varphi] \mid E[\varphi\ U\ \varphi]$, using some examples; the others $\bot \mid \top \mid P \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi)$ are self explanatory.

- *AXφ:* A state (s0 in example of Figure 4) is marked *AXφ* if in all paths starting from that state (s0) the next state satisfies φ.

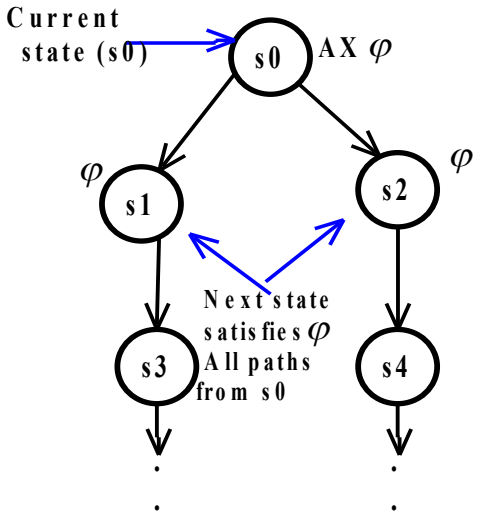


Figure 4. *AXφ* is satisfied in state *s0*

- *EXφ*: A state (s0 in example of Figure 5) is marked *EXφ* if there is at least one path starting from that state (s0) where the next state satisfies φ.

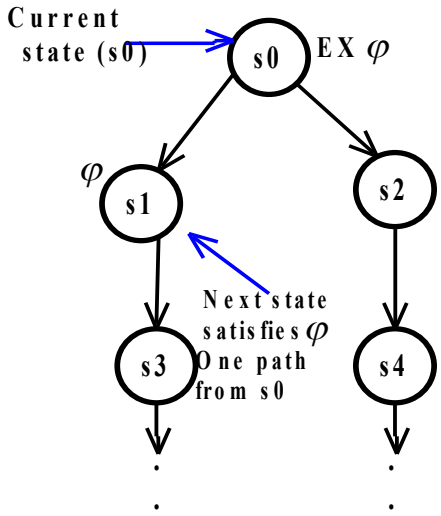


Figure 5. *EXφ* is satisfied in state *s0*

- *AFφ:* A state (s0 in example of Figure 6) is marked *AFφ* if in all paths from that state (s0) there is a state in future (s3 and s4 in the example) where φ is satisfied.

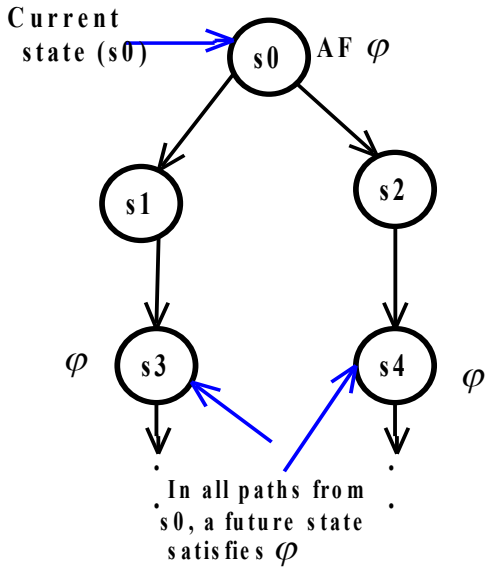


Figure 6. *AFφ* is satisfied in state *s0*

- *EFφ:* A state (s0 in example of Figure 7) is marked *EFφ* if there is a path from that state (s0) where there is a state in future (s3 and s4 in the example) where φ is satisfied.

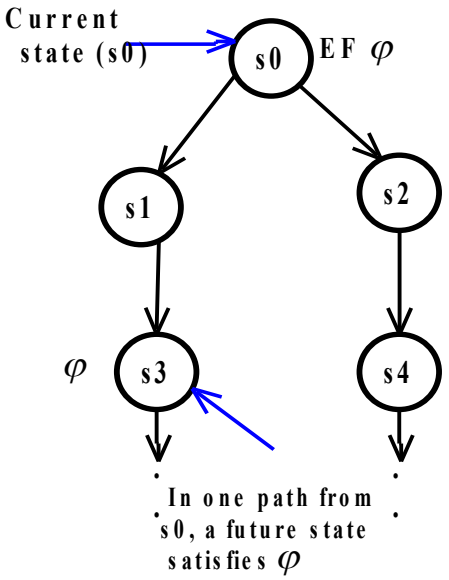


Figure 7. *EFφ* is satisfied in state *s0*

- *AGφ:* A state (s0 in example of Figure 8) is marked *AGφ* if in all paths from that state (s0) all states globally (s1,s2, s3 and s4 in the example) satisfy φ.

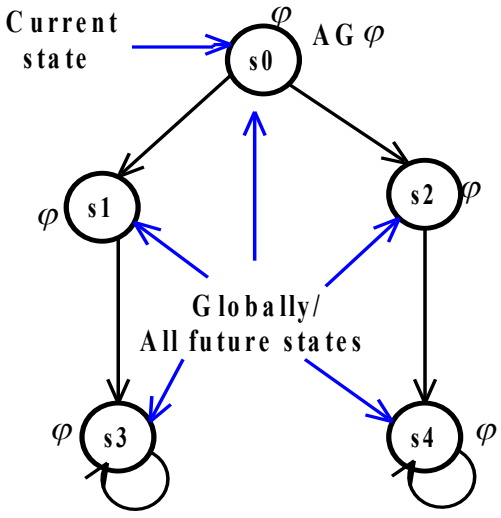


Figure 8. *AGφ* is satisfied in state *s0*

- *EGφ:* A state (s0 in example of Figure 9) is marked *EGφ* if there is a path from that state (s0) where all states globally (s2 and s4 in the example) satisfy φ.

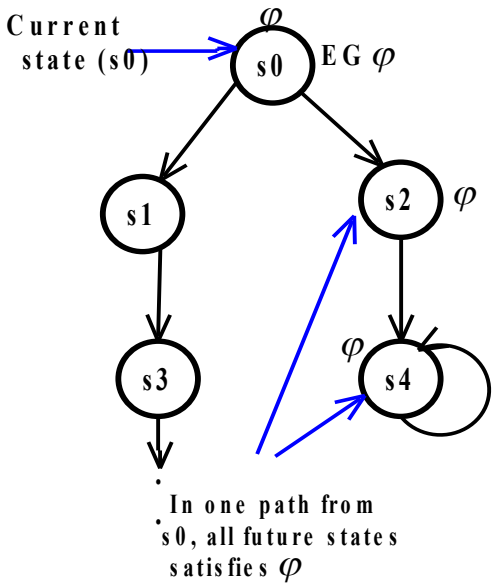


Figure 9. *EGφ* is satisfied in state *s0*

- *A[p U q]*: A state (s0 in example of Figure 10) is marked *A[p U q]* if all paths from that state (s0) satisfies the following:
  - states must satisfy *p* until a state comes that satisfy *q*

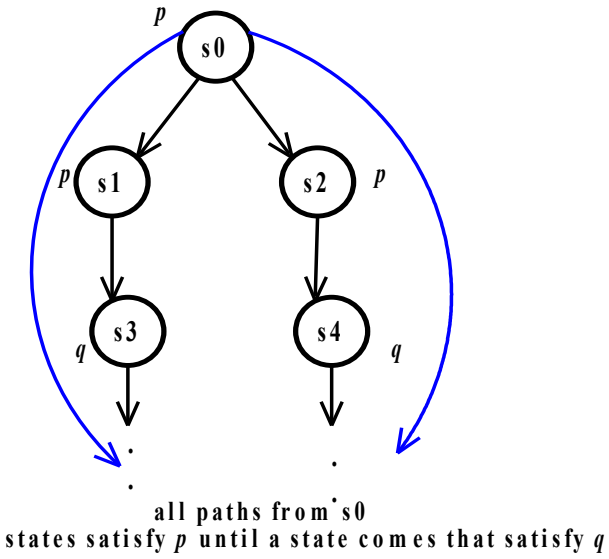


Figure 11. *A[p U q]* is satisfied in state *s0*

- *E[p U q]*: A state (s0 in example of Figure 10) is marked *E[p U q]* if there is at least one path from that state (s0) satisfies the following:
    - states must satisfy *p* until a state comes that satisfy *q*

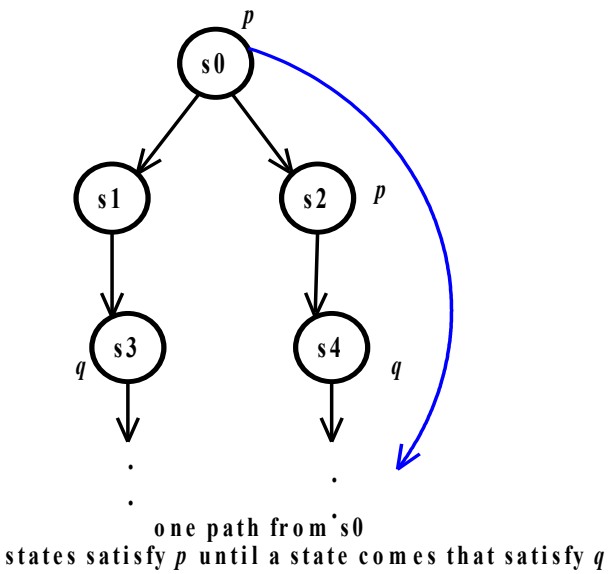


Figure 12. *E[p U q]* is satisfied in state *s0*

2.1 **Priorities of different connectives in CTL formulas**

The order of priorities of different connectives in CTL are as follows

- 1st: Unary connective and temporal connectives AG, EG, AF, EF, AX and EX
- 2nd: $\wedge$ and $\vee$,
- 3rd: $\rightarrow$, AU and EU.

We can use brackets in order to override these priorities.

This concept of propriety is illustrated by constructing a parse tree for the CTL formula $A[AX\ p\ U\ EX(\ p \wedge q)] \rightarrow A[p\ U\ q]$, which is shown in Figure 13.

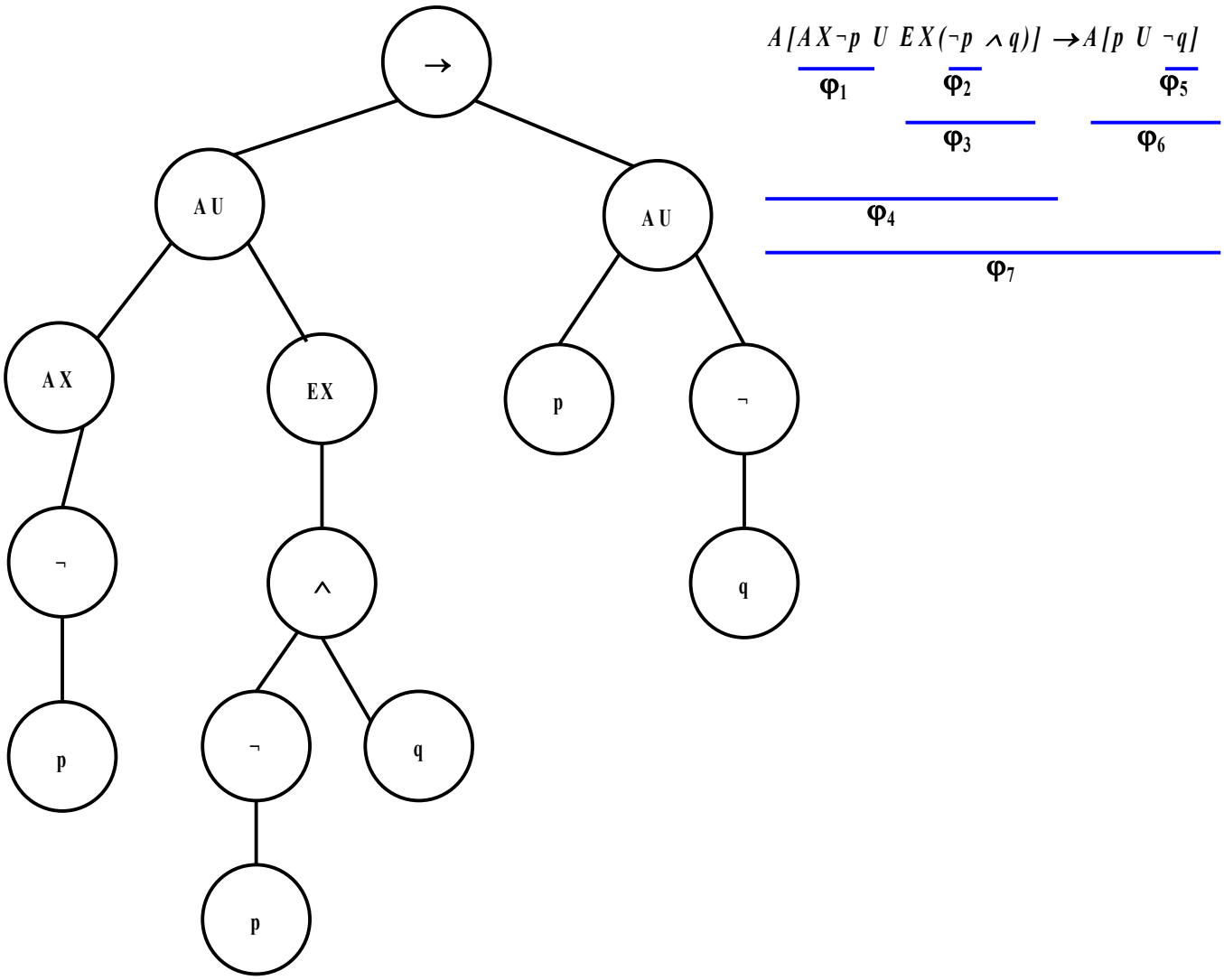


Figure 13. Parse tree for the CTL formula $A[AX\ p\ U\ EX(\ p \wedge q)] \rightarrow A[p\ U\ q]$

2.2 **Well-formed CTL formula**

Let us see some examples of well-formed CTL formulas and some examples which are not well-formed, in order to understand the syntax. Suppose that *p*, *q* and *r* are atomic formulas. The following are well-formed CTL formulas:

- *AGp | EGp | AXp | EXp | AFp |EFp | A[p U q] | E[p U q].*
- *AG(p ∧q).*
- *AG(p → EG q).*
- *EGp ∨ E(q U r).*
- *AG (p ∧ q)*
- *AG (EFp ∧ q).*
- *AFEGp .*
- *A[p U A[q U r]].*
- *A[AX p U EX(p ∧ q)] →A[p U q].*

The following are not well-formed formulas:

i. *Gp* : *G* must be preceded by a path quantifier (*A* or *E*).
ii. *EFGr:* Same reason as for formula (i)
iii. *F[r U q]:* Until operator must be preceded by a path quantifier (*A* or *E*).
iv. *AEFr:* Path quantifier A should be followed by a temporal connective *G,F* or *X*. For example if we write *AFEFr* or *AGEFr,* then the formula is well-formed.
v. *A[(r U q) ∧ (p U r)]:* Boolean connectives (like ∨ or ∧) cannot be directly inside path quantifier *A* or *E*. However we can write *A[(r ∨ q)U(p ∨ r)].*

3. **Semantics of CTL**

The semantics of CTL is defined over a model '*M'* , which is defined as 3-tuple $\langle S, \rightarrow, L \rangle$, where

- *S*: set of states of *M*.
- $\rightarrow$: set of transitions $T \subseteq S \, X \, S$ such that for every $s \in S$ there exists a state *s'* such that $s \rightarrow s'$. In other words, there is at-least one transition emanating from each state.
- *L:* labeling for each state and defined by $L:S \rightarrow \rho(P)$, *P* is set of atomic propositions and $\rho(P)$ is the power set of *P*.

This model is also known as Kripke structure. A Kripke structure is similar to a state transition diagram, which has states and transitions from one state to another, but includes two extra features, given as follows:

- All states must have at least one outgoing edge.
- Each state is labeled with one of the element of the power set of atomic propositions. These atomic propositions will be assigned to either true (T) or false (F) values.

Figure 14 shows two state transition diagrams (the labeling of the states is not shown, but assumes that all states have labels with a subset of atomic propositions). The left side diagram is not a Kripke structure because state s6 does not have an outgoing transition. The state transition diagram on the right side is a Kripke structure because it satisfies all the requirements of the model discussed above.

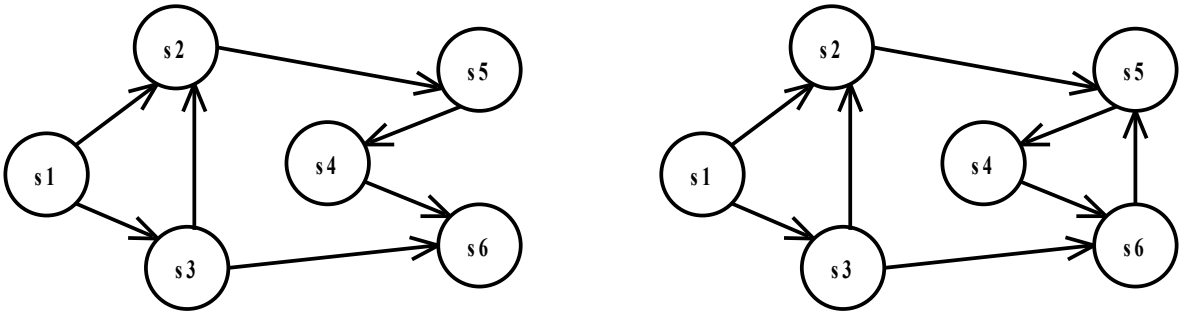


Figure 14. State transition diagrams: Left (Right) is an invalid (valid) Kripke structure

Semantics of CTL is defined by a relation $\models$ on state *s,* over formula φ on model *M*, which is written as "s $\models$ φ"—this implies CTL formula φ holds in state *s* of model *M*. CTL formulas are interpreted over Kripke structure after unfolding it as tree. Figure 15 shows a Kripke structure, where the set of atomic propositions is *P={p,q,r}*. The model in terms of the parameters discussed above is as follows

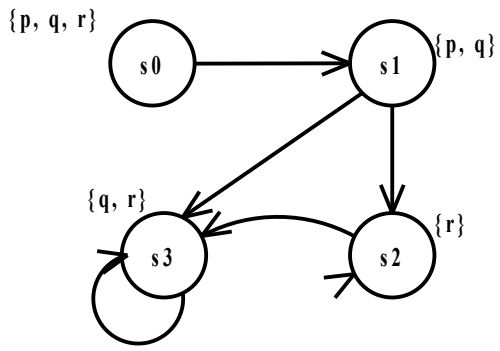


Figure 15. Example of a Kripke structure

- *S*={s0,s1,s2,s3}
- $\rightarrow$={{s0, s1}, {s1,s2}, {s1, s3}, {s2,s3}, {s3,s2},{s3,s3}}.
- L: L(s0)={p,q,r}, L(s1)={p, q}, L(s2)={r}, L(s3)={q, r}.

Figure 16 shows the tree of the Kripke structure of Figure 15, when it is unfolded over time.

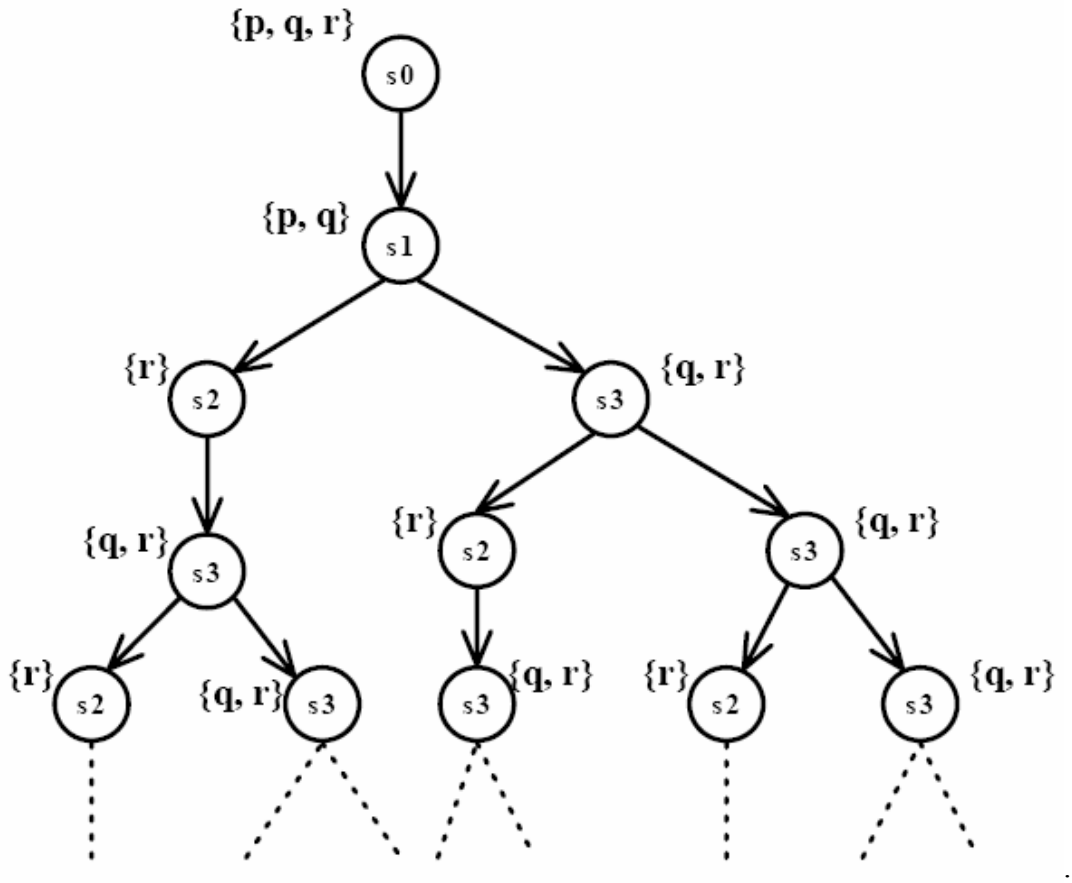


Figure 16. The unfolded tree of the Kripke structure of Figure

To determine whether $s \models \varphi$ on M holds, requires a recursive procedure on the structure of φ, which is discussed as follows:

- If φ is atomic, satisfaction is determined by L. For example, if φ is atomic p say, then its truth at a state s can be determined by looking at the labels at s.
- If the root in the parse tree of φ is a Boolean connective ($\wedge, \vee$, $\top$ etc.) then the satisfaction of $s \models \varphi$ is answered by the usual truth-table definition and further recursion down φ. For example, if φ is "$AX \quad p$" say, then its truth at a state s can be determined by truth table value of p and then evaluating AX on the result.
- If the top level connective of φ is a path operator beginning with *A (E)*, then satisfaction holds if all (there exists at least one) paths from s satisfy that formula resulting from removal of *A (E)*. If the result of removing A or E contain further As or Es, they will be dealt with by recursion.

The relation $s \models \varphi$ is defined by structural induction on φ, as follows

1. $M, s \models \top$ and $M, s \not\models \bot$; "truth is true and false is false in any state s"
2. $M, s \models p$ iff $p \in L(s)$; atomic proposition p is satisfied if label of s has p.
3. $M, s \models \quad \varphi$ iff $M, s \not\models \varphi$. φ is satisfied at s if s does not satisfy φ.

Figure 17 illustrates relations discussed in point 2 and point 3, when φ is p and p, respectively.

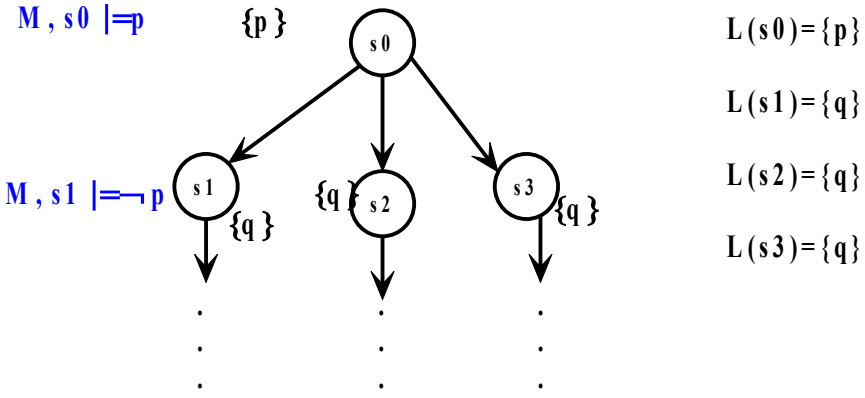


Figure 17. Illustration of relations discussed in point 2 and point 3

4. $M, s \models \varphi 1 \wedge \varphi 2$ iff $M, s \models \varphi 1$ and $M, s \models \varphi 2$; $\varphi 1 \wedge \varphi 2$ is satisfied at s if in s both $\varphi 1$ and $\varphi 2$ are satisfied.
5. $M, s \models \varphi 1 \vee \varphi 2$ iff $M, s \models \varphi 1$ or $M, s \models \varphi 2$; $\varphi 1 \vee \varphi 2$ is satisfied at s if in s either $\varphi 1$ or $\varphi 2$ is satisfied. Figure 18 illustrates relations discussed in point 4 and point 5.

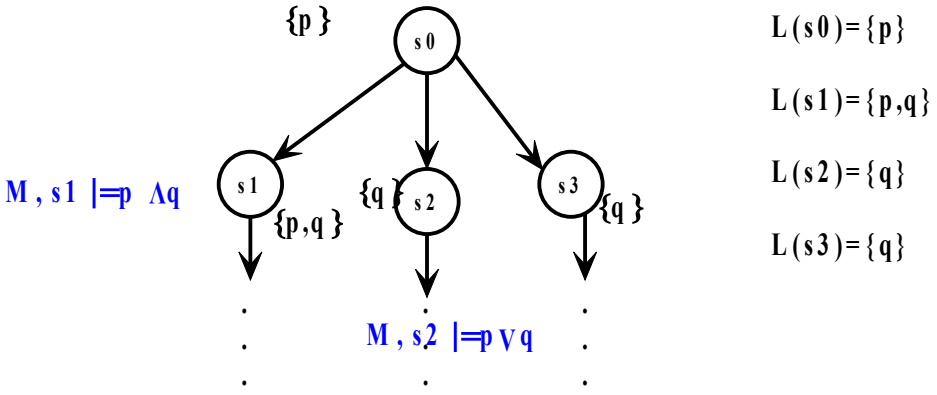


Figure 18. Illustration of relations discussed in point 4 and point 5

6. *M, s* $\models$ *φ1* → *φ2* iff *M, s* $\not\models$ *φ1* or *M, s* $\models$ *φ2; φ1* → *φ2* is satisfied at *s* if in *s* either *φ1* is not satisfied or *φ2* is satisfied. In Figure 18, we have s1 $\models$ *p* → *q*.

7. *M, s* $\models$ *AXφ* iff for all s1 such that s → s1, we have *M,* s1 $\models$ φ; *AXφ* is satisfied at *s* if in all next states of *s,* φ is satisfied. In Figure 18, we have s0 $\models$ *AXq*. Thus, *AX* says: 'in every next state.'

8. *M, s* $\models$ *EXφ* iff for one state s1 such that s → s1 we have *M,* s1 $\models$ φ; *EXφ* is satisfied at *s,* if in some next state of *s,* φ is satisfied. In Figure 18, we have s0 $\models$ *EXp*. Thus, *EX* says: 'in at least one next state.'

9. *M, s* $\models$ *AGφ* holds iff for all paths s1 → s2 → s3 → . . ., where *s*=s1, and all s_i along the path, *M,* $s_i \models \varphi$. *AGφ* is satisfied at *s* if all states of all paths emanating from *s* satisfies *φ*. In addition, in *s, φ* must hold. So, *AGφ* is "All computation paths beginning in *s* has the property φ holding "Globally". Figure 19 illustrates the relation discussed in this point.

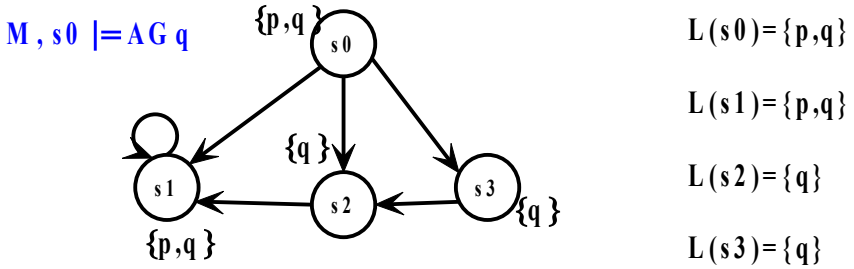


Figure 19. Illustration of the relation discussed in point 9

10. *M, s* $\models$ *EGφ* holds iff there is a path s1 → s2 → s3 → . . ., where *s*=s1, and all s_i along the path, *M,* $s_i \models \varphi$. *EGφ* is satisfied at *s* if all states of at least one path emanating from *s*

satisfies φ. In addition, in *s*, φ must hold. So, $EG\varphi$ is "At least one computation path beginning in *s* has the property φ holding "Globally". In Figure 19 state s0 satisfies *EGp* (the path s0 → s1 → s1 → ....).

11. *M, s* $\models AF\varphi$ holds iff for all paths s1 → s2 → s3 → . . ., where *s*=s1, and for at least one s_i along the path, *M,* $s_i \models \varphi$. $AF\varphi$ is satisfied at *s* if some "future" state of all paths emanating from *s* satisfies φ. So, $AF\varphi$ is "All computation paths beginning in *s* has the property φ holding "in Future". Figure 20 illustrates the relation discussed in this point.

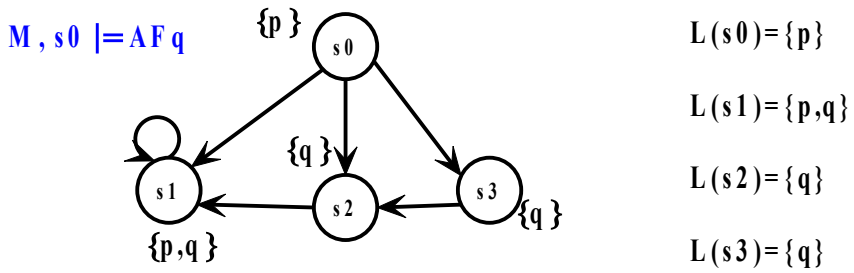


Figure 20. Illustration of the relation discussed in point 11

12. *M, s* $\models EF\varphi$ holds iff there is one path s1 → s2 → s3 → . . ., where *s*=s1, and for at least one s_i along the path, *M,* $s_i \models \varphi$. $EF\varphi$ is satisfied at *s* if some "future" state of at least one path emanating from *s* satisfies φ. So, $EF\varphi$ is "At least one computation path beginning in *s* has the property φ holding "in Future". Figure 21 illustrates the relation discussed in this point. It may be noted that here, from state s0, path s0 → s1 satisfies *EFp.*

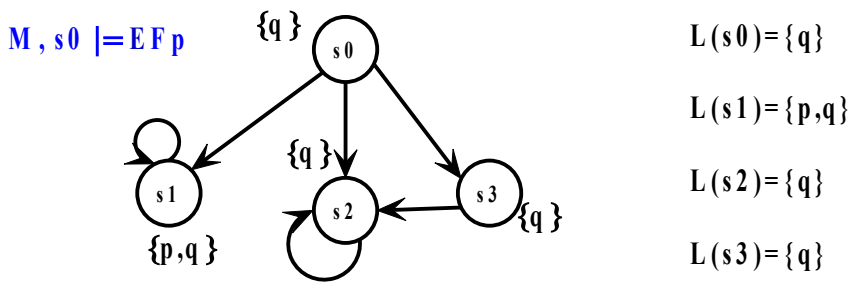


Figure 21. Illustration of the relation discussed in point 12.

13. *M, s* $\models$ *A[φ1 U φ2]* holds iff for all paths s1 → s2 → s3 → . . ., where *s*=s1, *φ1 U φ2* is satisfied, i.e., there is some s_i along the path, such that *M,* s_i $\models$ *φ2*, and for each $j < i$, we have *M,* s_j $\models$ *φ1* or along the path *M,* s_j $\models$ *φ1* for all $0 \le j$. So, *A[φ1 U φ2]* is "All computational paths beginning in *s* has the property *φ1 U φ2*. Figure 22 illustrates the relation discussed in this point.

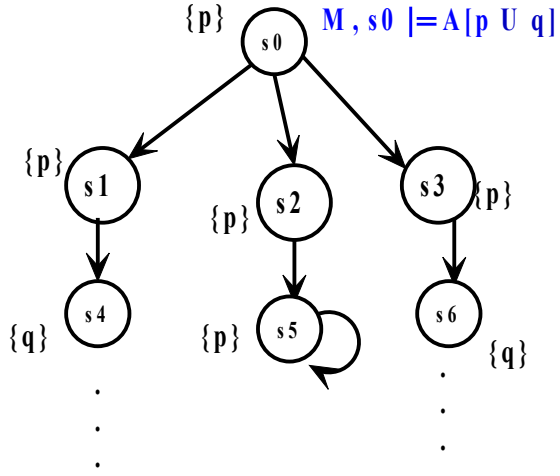


Figure 22. Illustration of the relation discussed in point 13.

14. *M, s* $\models$ *E[φ1 U φ2]* holds iff for at least one path s1 → s2 → s3 → . . ., where *s*=s1, *φ1 U φ2* is satisfied, i.e., there is some s_i along the path, such that *M,* s_i $\models$ *φ2*, and for each $j < i$, we have *M,* s_j $\models$ *φ1* or along the path *M,* s_j $\models$ *φ1* for all $0 \le j$. So, *E[φ1 U φ2]* is "At least one computational path beginning in *s* has the property *φ1 U φ2*. Figure 23 illustrates the

relation discussed in this point. In (only the path) s0 → s1 → s4 →….*p U q* is satisfied, making *so* ⊨ *E[p U q]*

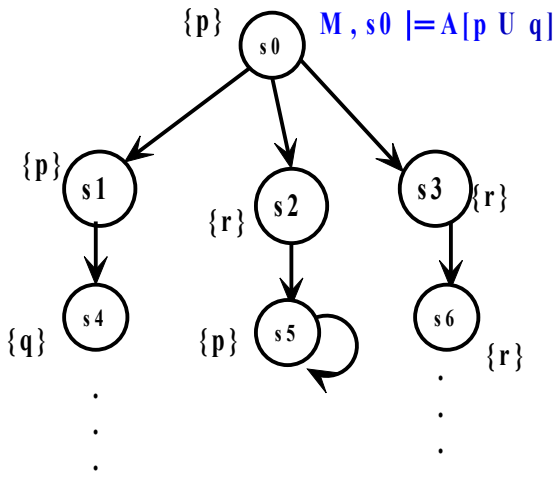


Figure 23. Illustration of the relation discussed in point 14.

4. **Conclusions**

In this lecture we have introduced the syntax and semantics of CTL. We have discussed how well-formed CTL formulas can be construed and how to determine if they are satisfied in a state of the corresponding model. Just like we can write $a \; OR \; b \equiv \overline{\overline{a} \text{ AND } \overline{b}}$

, similarly one CTL formula can be represented in terms of another equivalent CTL formula. In the next lecture we will discuss important equivalences of CTL formulas.

Question and Answers

Question: Assume that atomic propositions are three words “started”, “busy” and “requested”. Encode the following two statements in CTL.

1. It is possible to go to a state where a process has started but is not ready (i.e., still busy) to deliver the result
2. For any state, if request (of some resource) occurs, then it will eventually be started.

Answer:

1. *EF(started ^ busy)*
2. *AG(requested → AF started)*